

Vježba — Izrada User Search aplikacije

U ovoj vježbi izradit ćemo malu React aplikaciju koja dohvaća korisnike s javnog API-ja i omogućuje pretraživanje korisnika po imenu.

Ova vježba povezuje nekoliko važnih React koncepata:

- `useEffect`
 - dohvaćanje podataka pomoću `fetch`
 - dependency array `[]`
 - `loading` state
 - `error` state
 - spremanje podataka u state
 - kontrolirani input
 - filtriranje arraya
 - uvjetno renderiranje
 - prikaz liste pomoću `.map()`
-

Cilj vježbe

Cilj je napraviti komponentu `UserSearch` koja:

1. kada se komponenta prvi put prikaže, dohvaća listu korisnika s API-ja
 2. prikazuje poruku "`Loading users...`" dok se podaci dohvaćaju
 3. prikazuje poruku greške ako request ne uspije
 4. prikazuje input za pretraživanje kada su podaci dostupni
 5. filtrira korisnike prema imenu dok korisnik tipka
 6. prikazuje poruku ako nema rezultata pretrage
 7. prikazuje ime i email svakog korisnika
-

Naziv projekta

08_React/07_Practice_UserSearch/

Scenarij

Zamislite da izrađujete malu aplikaciju za prikaz korisničkog direktorija.

Aplikacija treba dohvatiti korisnike s javnog API-ja:

<https://jsonplaceholder.typicode.com/users>

Nakon što se korisnici dohvate, korisnik aplikacije treba moći upisati tekst u search box. Lista korisnika se zatim treba automatski filtrirati prema imenu.

Na primjer, ako korisnik upiše:

Leanne

aplikacija treba prikazati samo korisnika čije ime sadrži taj tekst.

Zadatak

1. Kreirajte novi React projekt

U terminalu pokrenite:

```
npm create vite@latest user-search-app -- --template react
```

Zatim uđite u projekt:

```
cd user-search-app
```

Instalirajte pakete:

```
npm install
```

Pokrenite aplikaciju:

```
npm run dev
```

2. Očistite početni projekt

U datoteci `src/App.jsx` obrišite početni sadržaj.

Pripremite osnovnu komponentu:

```
import './App.css';
```

```
function App() {
```

```
return (  
  <main className="app">  
    <h1>User Search App</h1>  
  </main>  
);  
}
```

export default App;

U datoteci `src/App.css` obrišite postojeći CSS.

3. Napravite komponentu **UserSearch**

U folderu `src` napravite novu datoteku:

UserSearch.jsx

U njoj pripremite osnovnu komponentu:

```
function UserSearch() {  
  return (  
    <section>  
      <h2>User Directory</h2>  
    </section>  
  );  
}
```

export default UserSearch;

Zatim u `App.jsx` importajte i prikažite komponentu:

```
import UserSearch from "./UserSearch";  
import "./App.css";
```

```
function App() {  
  return (  
    <main className="app">  
      <h1>User Search App</h1>  
      <UserSearch />  
    </main>  
  );  
}
```

export default App;

4. Dodajte stateove

U `UserSearch.jsx` importajte `useState` i `useEffect`:

```
import { useState, useEffect } from "react";
```

Zatim unutar komponente dodajte četiri state varijable:

```
const [users, setUsers] = useState([]);
const [search, setSearch] = useState("");
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
```

Ove varijable imaju jasne uloge.

`users` čuva listu korisnika koju dobijemo s API-ja.

`search` čuva tekst koji korisnik upisuje u search input.

`loading` govori je li request još uvijek u tijeku.

`error` čuva poruku greške ako request ne uspije.

5. Dohvatite korisnike pomoću `useEffect`

Dodajte `useEffect` koji se izvršava samo jednom, nakon početnog renderiranja.

```
useEffect(() => {
  fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => {
      if (!response.ok) {
        throw new Error("Failed to fetch users");
      }

      return response.json();
    })
    .then(data => {
      setUsers(data);
      setLoading(false);
    })
    .catch(err => {
      setError(err.message);
      setLoading(false);
    });
}, []);
```

Važno je da dependency array bude prazan:

```
[]
```

To znači da se effect izvršava samo jednom, kada se komponenta prvi put prikaže.

Ovdje je posebno važno razumjeti zašto `fetch` ne pišemo direktno u tijelo komponente. Kada bismo to napravili, request bi se pokretao na svakom renderiranju, a `setUsers` bi ponovno pokrenuo renderiranje. To može dovesti do beskonačnog ciklusa.

Ispravan obrazac je:

render → useEffect → fetch → setUsers → re-render

6. Dodajte loading i error prikaz

Prije glavnog `return` dijela dodajte uvjetno renderiranje:

```
if (loading) {  
  return <p>Loading users...</p>;  
}
```

```
if (error) {  
  return <p>Error: {error}</p>;  
}
```

Ako su podaci još u procesu dohvaćanja, nema smisla prikazivati listu korisnika.

Ako postoji greška, aplikacija treba prikazati poruku greške.

7. Dodajte kontrolirani search input

U JSX dodajte input:

```
<input  
  type="text"  
  value={search}  
  onChange={event => setSearch(event.target.value)}  
  placeholder="Search by name..."  
>
```

Ovo je kontrolirani input zato što njegova vrijednost dolazi iz statea:

```
value={search}
```

a svaka promjena ažurira state:

```
onChange={event => setSearch(event.target.value)}
```

Kada korisnik tipka, `search` state se mijenja i komponenta se ponovno renderira.

8. Filtrirajte korisnike

Prije `return` dijela, ali nakon `loading` i `error` provjera, napravite novu varijablu:

```
const filteredUsers = users.filter(user =>
  user.name.toLowerCase().includes(search.toLowerCase())
);
```

Ovdje uzimamo originalni `users` array i vraćamo samo korisnike čije ime sadrži tekst iz `search` inputa.

Koristimo `toLowerCase()` na obje strane kako pretraga ne bi ovisila o velikim i malim slovima.

Na primjer, korisnik može upisati:

leanne

i aplikacija i dalje može pronaći:

Leanne Graham

9. Prikažite listu korisnika

Ako `filteredUsers` nema nijedan element, prikažite:

No users match your search.

Ako postoje rezultati, prikažite ih u listi:

```
{filteredUsers.length === 0 ? (
  <p>No users match your search.</p>
) : (
  <ul className="user-list">
    {filteredUsers.map(user => (
      <li key={user.id} className="user-item">
        <strong>{user.name}</strong> - {user.email}
      </li>
    ))}
  </ul>
)}
```

```
</ul>
})
```

Ovdje koristimo `.map()` za prikaz liste korisnika.

Svaki `<li>` mora imati `key`.

Koristimo:

```
key={user.id}
```

jer svaki korisnik s API-ja ima jedinstveni `id`.

Očekivano ponašanje aplikacije

Kada se aplikacija otvori, najprije se prikazuje:

Loading users...

Kada se podaci učitaju, prikazuje se:

User Directory
Search by name...
lista korisnika

Kada korisnik upiše tekst u search input, lista se odmah filtrira.

Ako nema korisnika koji odgovaraju pretrazi, prikazuje se:

No users match your search.

Ako API request ne uspije, prikazuje se poruka greške.

Puno rješenje

`src/App.jsx`

```
import UserSearch from "../UserSearch";
import "../App.css";
```

```
function App() {
  return (
```

```
<main className="app">
  <h1>User Search App</h1>
  <UserSearch />
</main>
);
}
```

```
export default App;
```

src/UserSearch.jsx

```
import { useState, useEffect } from "react";

function UserSearch() {
  const [users, setUsers] = useState([]);
  const [search, setSearch] = useState("");
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(response => {
        if (!response.ok) {
          throw new Error("Failed to fetch users");
        }

        return response.json();
      })
      .then(data => {
        setUsers(data);
        setLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setLoading(false);
      });
  }, []);

  if (loading) {
    return <p>Loading users...</p>;
  }

  if (error) {
    return <p>Error: {error}</p>;
  }
}
```



```

const filteredUsers = users.filter(user =>
  user.name.toLowerCase().includes(search.toLowerCase())
);

return (
  <section className="user-search">
    <h2>User Directory</h2>

    <input
      className="search-input"
      type="text"
      value={search}
      onChange={event => setSearch(event.target.value)}
      placeholder="Search by name..."
    />

    {filteredUsers.length === 0 ? (
      <p>No users match your search.</p>
    ) : (
      <ul className="user-list">
        {filteredUsers.map(user => (
          <li key={user.id} className="user-item">
            <strong>{user.name}</strong> - {user.email}
          </li>
        ))}
      </ul>
    )}
  </section>
);
}

export default UserSearch;

```

src/App.css

```

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #eef2f7;
}

.app {
  max-width: 640px;
  margin: 50px auto;
  padding: 32px;
  background-color: white;
}

```

```
border: 1px solid #d8dee9;  
border-radius: 16px;  
}
```

```
h1,  
h2 {  
  text-align: center;  
}
```

```
.user-search {  
  margin-top: 24px;  
}
```

```
.search-input {  
  width: 100%;  
  box-sizing: border-box;  
  padding: 10px;  
  margin-bottom: 18px;  
  border: 1px solid #ccd3df;  
  border-radius: 8px;  
  font-size: 16px;  
}
```

```
.user-list {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
}
```

```
.user-item {  
  padding: 12px;  
  border-bottom: 1px solid #e4e8f0;  
}
```

```
.user-item:last-child {  
  border-bottom: none;  
}
```